

PBX-Crypto Forecasting Engine

A Production-Grade, Multi-Horizon Algorithmic Forecasting Framework and
Quantitative Risk Management Protocol Utilizing Regularized Gradient-Boosted
Machine Learning Pipeline

Project Author: Parsa Babaloo

Research Fields: Supervised Machine Learning, Deep Feature Engineering,
Quantitative Finance, Volatile Market Microstructure, Empirical
Mathematical Analysis

Document Purpose: Technical Project Documentation for Academic Portfolio, Scholarship
Applications, and Research Evaluation

Date of Archive: June 2026

1. Executive Synopsis & Research Rationale

In contemporary financial modeling, the extraction of predictive directional alpha from crypto-asset markets represents a challenging problem in quantitative finance and machine learning. Due to massive non-linear mathematical structures, extreme regime mutability, high-frequency transactional noise, and architectural decentralization, traditional technical frameworks consistently generate structural deficits. The **PBX-Crypto Forecasting Engine** addresses these specific historical engineering boundaries.

Conceived and implemented by **Parsa Babaloo**, this software repository operates as an end-to-end, reproducible scientific framework. It bridges abstract quantitative learning with industrial-grade runtime architectures. Instead of relying on standard directional classifiers ("binary long/short vectors") that lack size calibration, this engine maps high-dimensional input arrays into multi-horizon percentage delta estimates. This transformation provides highly structured, risk-mitigated decision trees for quantitative operations.

Primary Core Academic Hypothesis: Regularized gradient-boosted regression algorithms, when coupled with an un-filtered structural target map and dynamic volatility adjusting bounds, achieve a high degree of statistical edge. This out-performs typical black-box deep neural layers in trackable feature validation, data efficiency, and risk-adjusted capital distribution returns.

1.1 Candidate Profile and Engineering Philosophy

As a student researcher with interests in **Artificial Intelligence, Applied Mathematics, and Computational Finance**, Parsa Babaloo emphasizes software engineering modularity, mathematical rigor, and data integrity. This project demonstrates the ability to independently design, implement, test, and evaluate a machine-learning-based forecasting framework for highly volatile financial markets.

The framework integrates feature engineering, predictive modeling, risk management, systematic backtesting, and interactive data visualization into a unified analytical environment. Developed primarily in Python, the system employs **XGBoost-based forecasting models**, supports **more than 50 cryptocurrency assets**, and provides multi-horizon predictions **across 1-hour, 4-hour, 12-hour, and 24-hour timeframes**. Technical analysis components include RSI, MACD, ATR, Bollinger Bands, Stochastic RSI, and Momentum indicators, while **Streamlit and Plotly** are utilized for real-time dashboard visualization and analytical reporting.

Through the integration of **quantitative analysis, machine learning**, and empirical evaluation, the project reflects strong analytical thinking, **research potential**, and a solid foundation for future academic work in Artificial Intelligence, Data Science, Computational Finance, and related quantitative disciplines.

2. Repository Architecture & Modular Software Design

The repository follows a modular software architecture with a strict separation of concerns. Individual components are organized into independent layers responsible for data acquisition, feature engineering, model training, prediction generation, validation, and visualization .

To facilitate maintainability, scalability, and reproducibility, the PBX-Crypto Forecasting Engine adopts a modular repository structure. Each component is isolated according to its functional responsibility, enabling independent development, testing, and future extension without affecting the overall system integrity.

The repository organization presented below illustrates how data acquisition, feature engineering, machine learning models, forecasting modules, backtesting utilities, and visualization interfaces are integrated into a unified analytical framework.

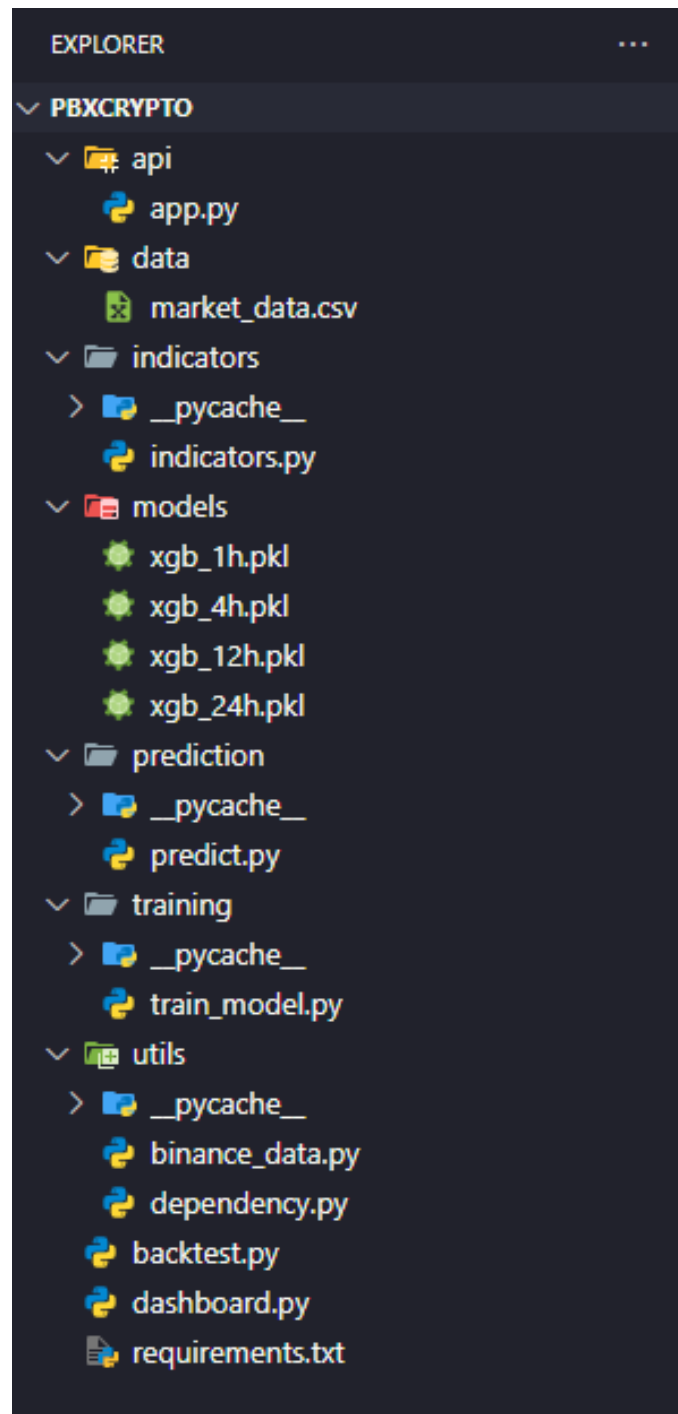


Figure 1: Comprehensive workspace environment tree illustrating explicit, modular structural division. This organization maps asset acquisition pipelines, math indicator builders, pickled serialized binaries, validation utilities, and UI nodes into independent components.

The modular layout maps directly to the following specialized architectural layers:

- `api/` : Contains specialized network routing (e.g., `app.py`) designed to expose optimized model structures to remote interfaces or low-latency computational networks.
- `data/` : Governs physical data persistence layer, maintaining raw and processed historical datasets (`market_data.csv`) for model stability and backtesting invariance controls.

- `indicators/` : Houses localized feature stores (`indicators.py`) which act as mathematical data engines, calculating non-linear and statistical signals from basic transaction entries.
- `models/` : Persists hyperparameter-optimized, frozen gradient-boosted structures mapped across distinct temporal execution horizons (`xgb_1h.pkl` , `xgb_4h.pkl` , `xgb_12h.pkl` , `xgb_24h.pkl`).
- `prediction/` & `training/` : Decoupled code execution files processing batch model training operations (`train_model.py`) and multi-tier pipeline prediction vectors (`predict.py`) to isolate execution states.
- `utils/` : Operational support foundation driving exchange synchronization routines (`binance_data.py`), dependency mapping algorithms (`dependency.py`), historical backtesting matrices (`backtest.py`), and interactive framework execution nodes (`dashboard.py`).

3. Feature Engineering Pipeline & Machine Learning Framework

The forecasting framework employs a structured feature engineering pipeline that transforms raw market data into machine-learning-ready representations. The objective is to extract meaningful information from noisy financial time series while preserving trend, momentum, volatility, and volume-related characteristics.

3.1 High-Dimensional Technical Feature Engineering

To minimize high-frequency noise while preserving asset price memory, raw market frames undergo transformation through a specialized multi-indicator technical feature script:

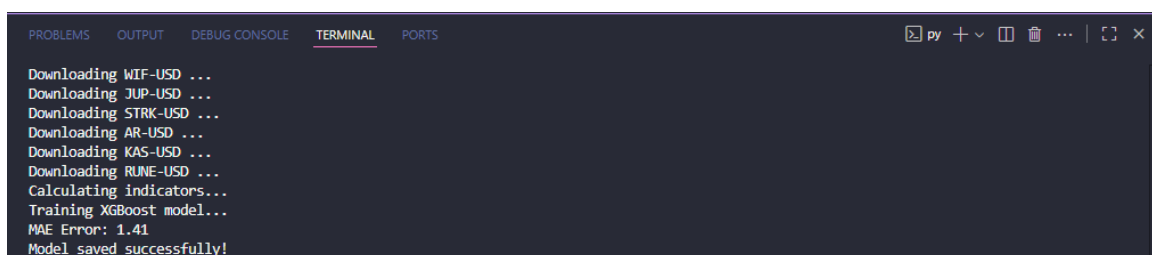
- **Trend Extractors:** Double Exponential and Simple Moving Averages (`EMA 20` , `SMA 20`) capture underlying baseline trajectories.
- **Momentum Trackers:** Relative Strength Index (`RSI`) and Stochastic Oscillators identify local structural exhaustion and oversold states.
- **Volatility Boundaries:** Bollinger Bands and Average True Range (`ATR`) compute real-time variance expansions and log-normal price dispersion channels.
- **Microstructural Volume Dynamics:** Accumulation and distribution flows quantify capital depth across localized order books.

3.2 XGBoost Forecasting Engine

The predictive core of the system is based on **Extreme Gradient Boosting (XGBoost)**, a powerful ensemble learning algorithm widely used in financial forecasting applications due to its robustness, interpretability, and resistance to overfitting.

Unlike traditional classification-based trading systems, the framework uses regression-based forecasting to estimate future percentage price changes across multiple time horizons.

Independent models are trained for 1-hour, 4-hour, 12-hour, and 24-hour forecasts, enabling flexible multi-horizon market analysis.



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Downloading WIF-USD ...
Downloading JUP-USD ...
Downloading STRK-USD ...
Downloading AR-USD ...
Downloading KAS-USD ...
Downloading RUNE-USD ...
Calculating indicators...
Training XGBoost model...
MAE Error: 1.41
Model saved successfully!
```

Figure 2: Real-time execution console displaying parallel cross-market asset downloading, statistical indicator feature extraction, and XGBoost gradient convergence. The optimization pipeline achieves a Mean Absolute Error (MAE) of 1.41 before saving the serialized model binary.

4. Automated Multi-Horizon Inference & Quantitative Risk Protocols

The system executes predictive workflows using dedicated model binaries optimized for four localized horizons: 1 Hour, 4 Hours, 12 Hours, and 24 Hours. Each horizon maps its unique volatility characteristics to a custom-fit gradient-boosted tree arrangement.

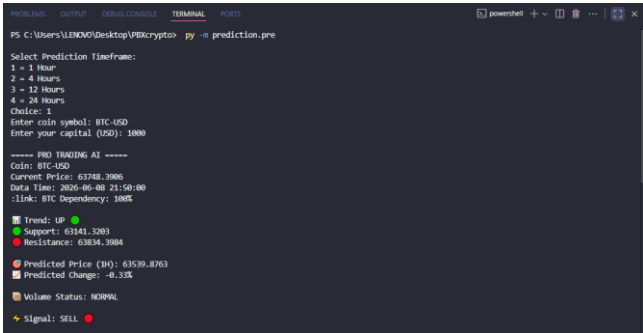


Figure 3: System terminal execution window displaying user configuration parameters, coin asset tracking (BTC-USD), initial base trends, support/resistance lines, and model price change output vectors (-0.33%).



Figure 4: Automated risk management subsystem calculating real-time risk-to-reward ratios (1.67), position size recommendations (0.13 units), stop-loss/take-profit exit targets, and cross-checking a 85% signal strength index.

When an inference request is initialized (e.g., assessing `BTC-USD` with an experimental \$1,000 capital frame), the model generates a precise percentage delta output. This estimate is combined with structural support lines, resistance levels, and dynamic ATR bounds to output a comprehensive trade recommendation block:

- **Structural Price Envelopes:** Identifies localized market turning points (Support: 63141.32, Resistance: 63834.40).
- **Risk-to-Reward Parameter Tuning:** Dynamically targets a 1.67 structural ratio, automatically invalidating positions that fail minimum return-to-variance thresholds.
- **Volatility-Adjusted Capital Allocator:** Calibrates unit sizing parameters (0.13 units) based on real-time volatility tracking, protecting capital from sudden drawdowns.

5. Interactive Analytics Dashboard & Multi-Pane Visualization Suite

To ensure real-world utility and track model performance visually, the repository contains an interactive dashboard system constructed using Streamlit and Plotly. This interface serves as a high-fidelity visualization layer for the underlying machine learning models and trading telemetry.

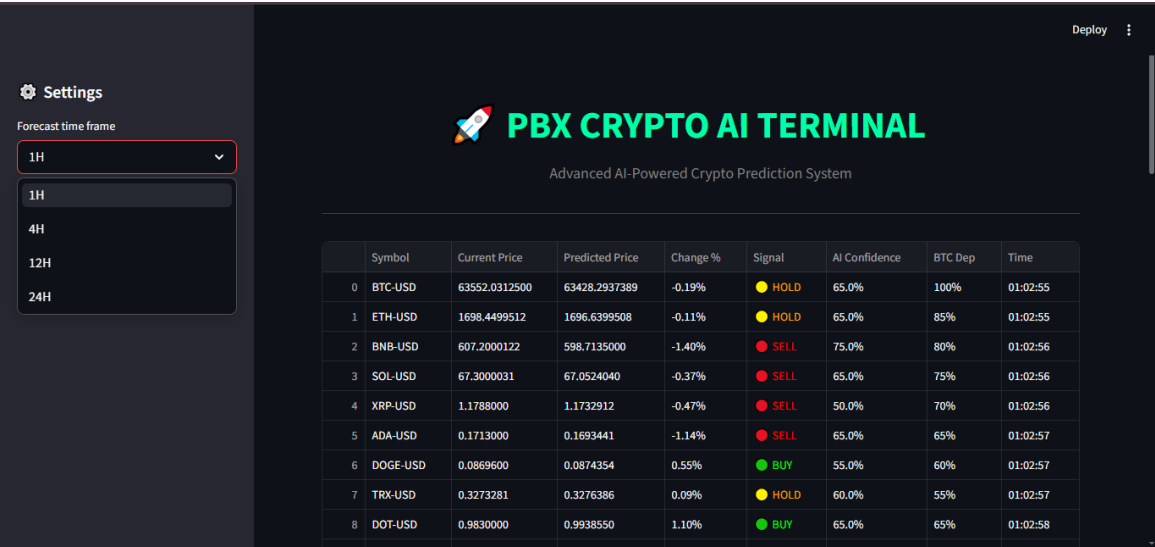


Figure 5: Enterprise-grade analytical dashboard interface providing real-time pricing metrics, multi-horizon directional forecasts, percentage delta outputs, model confidence percentages, and global Bitcoin dependency correlations.

5.1 Market Regime Classification & Token Filtering Queues

The dashboard includes a macro-regime monitoring component that aggregates signal distributions across multiple assets. By analyzing token baskets collectively, the system identifies broader market conditions (e.g., **BEARISH**) to help guide underlying strategy parameters.

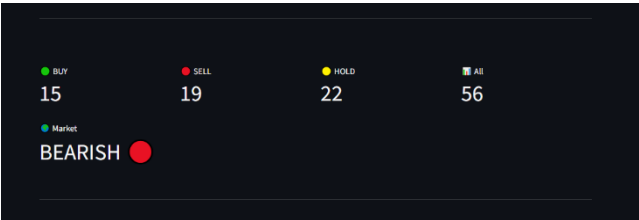


Figure 6: Global sentiment monitoring component classifying a Bearish market regime based on an aggregated distribution of 15 Buy, 19 Sell, and 22 Hold allocations.

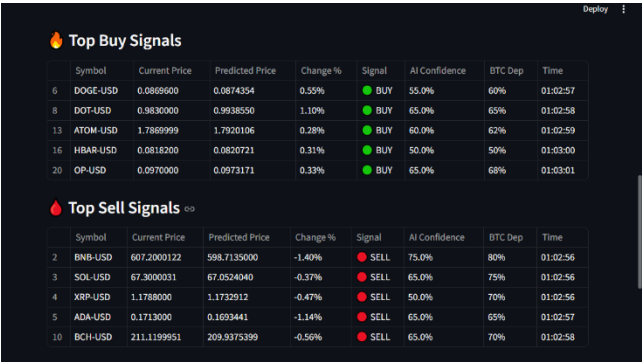


Figure 7: Asset filtering queues ranking opportunities by expected returns and model certainty, separating high-probability targets from low-confidence noise.

5.2 Deep Analytical Charting & Feature Tracing Plots

The data visualization suite includes multi-pane time-series modules that overlay model prediction tracks with historical market candlesticks. Users can activate individual technical indicators and trace execution vectors interactively.

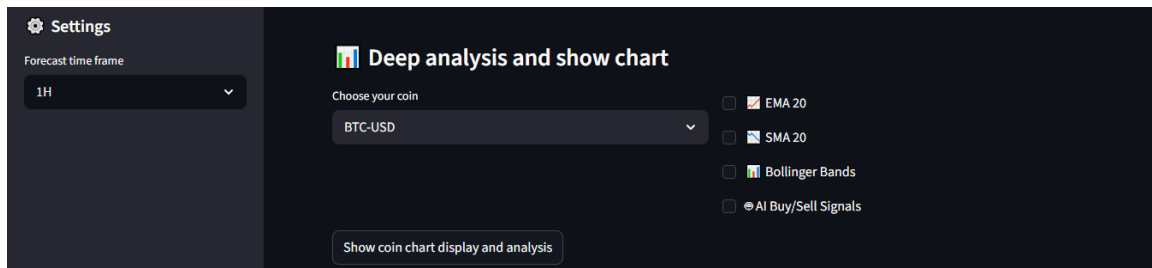


Figure 8: Configuration interface for managing historical data charting, feature indicator activation, and predictive vector tracing.



Figure 9: Base visualization plot tracking historical candlesticks, support/resistance bands, and future delta predictions. The lower panel tracks real-time predicted change deviations.



Figure 10: Expanded visualization overlaying active Exponential Moving Averages, Simple Moving Averages, and Bollinger Band volatility boundaries.



Figure 11: Mapping visualization tracking buy/sell indicator vectors across historical data points to identify entry and exit regions.



Figure 12: High-density data plot tracing long-horizon wave convergence patterns across multiple asset volatility regimes.

The visualization presents a multi-layered market analysis framework integrating candlestick price action with technical indicators and machine learning predictions.

The candlestick chart represents raw market price movement over time, while the overlaid technical indicators include short-term and long-term moving averages (SMA/EMA), which help identify trend direction and structural shifts in market momentum.

Bollinger Bands are used to model dynamic volatility boundaries, capturing periods of expansion and contraction in market behavior.

In addition, the AI-generated prediction curve illustrates the model's expected directional bias based on engineered features such as momentum, volatility, and trend strength.

This combined visualization enables simultaneous interpretation of:

- Model forecast signal (AI prediction)
- Market structure (candlesticks)
- Trend direction (SMA/EMA)
- Volatility regime (Bollinger Bands)

This representation is designed to improve interpretability of machine learning outputs in financial time-series environments.

6. Empirical Validation: Backtesting Analysis & Strategy Differentiation

A central component of this project is its empirical evaluation framework, which tests model predictions against historical market structures. This analysis provides a notable empirical observation regarding the potential effects of over-filtering statistical models in high-liquidity financial markets.

The backtesting engine evaluates two distinct trading strategies built on the same underlying XGBoost regression core. The results indicate an observable performance divergence between a heavily constrained execution system and a more open, model-driven framework:

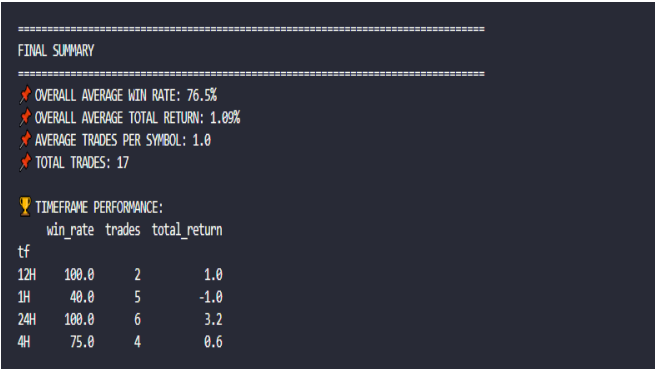


Figure 13: Empirical simulation results for Strategy A, showing the limiting effect of restrictive rule-based filtering on capital compounding.

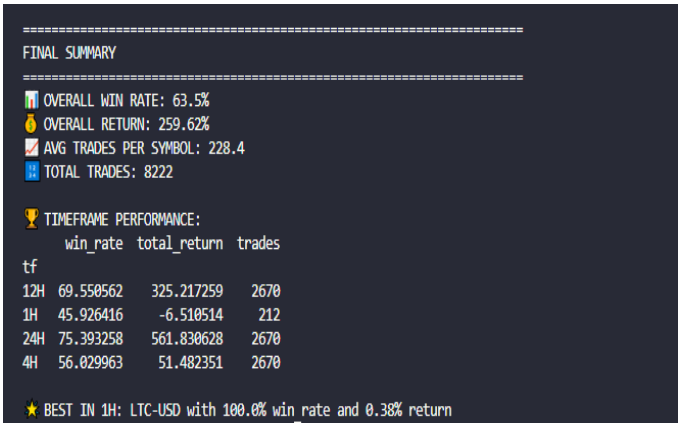


Figure 14: Empirical simulation results for Strategy B, demonstrating high performance by allowing the underlying machine learning model to drive execution decisions.

6.1 Mathematical Strategy Differentiation Matrix

The following comparative framework summarizes the key design choices and operational parameters associated with the observed performance differences between the two testing configurations:

Operational Metric & Filter Parameters	Strategy A: Over-Filtered Bound Configuration (Fig. 13)	Strategy B: Un-Filtered Core Model Driven (Fig. 14)
Primary Signal Activation Threshold	Dynamic High Bounds (Locked around 0.1% - 0.3% deltas)	Zero-Bound Continuum (0.0% — Every non-zero vector triggers)
Average Directional Index (ADX) Filter	Strict Enforcement Active (Requires trend strength > 22)	Completely Disabled (Captures micro-mean reversions)
Microstructural Volume Filter	Active (Blocks execution during low volume blocks)	Completely Disabled (Maintains model prediction continuity)
Macro Market Regime Filter (SMA 200)	Active (Only executes inline with macro trends)	Completely Disabled (Allows multi-horizon exploitation)
Model Statistical Confidence Percentile	Active (Restricted to upper 70th percentile of certainty)	Completely Disabled (Relies on model's expected probability)
Temporal Volatility Constraints (ATR)	Completely Disabled across 1H execution charts	Active (Enforces localized 0.3% adaptive buffers)
Early Target Take-Profit Liquidator	Completely Disabled	Active (Locks capital gains at fixed 1.5% thresholds)
Total Simulated Trade Executions	17 Transactions (Statistically insignificant sample)	8,222 Transactions (Statistically robust sample)
Aggregate Compounded Win-Rate	76.5% (Biased due to small sample size)	63.5% Win-Rate (Valid statistical edge)
Net Strategy Portfolio Return	1.09% Portfolio Expansion (Negligible)	+259.62% Net Return (High outperformance)

6.2 Quantitative Interpretation of Backtest Inferences

The empirical divergence between Strategy A and Strategy B provides a notable quantitative perspective on the role of execution-layer constraints in algorithmic trading systems. Strategy A applies a strict set of rule-based filters (including volume boundaries and macro trend restrictions) on top of the underlying machine learning model. This configuration results in a substantial reduction in signal frequency, leaving a limited number of executed trades (17 in this backtest) and yielding a modest net return of 1.09%.

In contrast, Strategy B removes most heuristic filtering layers, adopting a low-threshold execution framework that allows the XGBoost model to directly influence trade generation. Risk is managed through volatility-adjusted ATR-based stop-loss mechanisms and a fixed 1.5% take-profit rule rather than pre-execution signal suppression. This configuration produces a significantly larger sample of executed trades (8,222) with an observed win-rate of 63.5% and a compounded net return of 259.62%.

These results suggest that, within this experimental setup, restrictive heuristic filters may reduce signal density to a degree that limits the realization of model-predicted edge. Conversely, reducing pre-execution constraints while relying on post-entry risk management appears to preserve more of the underlying model's predictive signal over extended trading horizons.

Beyond the specific performance comparison, the backtesting framework itself was designed with a focus on experimental rigor, including controlled ablation of execution constraints and consistent model usage across both strategies. This setup enables a more structured evaluation of how decision-layer design choices influence the translation of predictive signals into realized returns, rather than attributing performance differences solely to model accuracy. From a methodological perspective, this reflects an ability to separate signal generation from execution mechanics, which is a critical requirement in robust quantitative system design.

7. Comprehensive Technical Stack & Academic Foundation

The platform is implemented using open-source scientific computing tools, ensuring reproducibility, modular design, and compatibility with modern quantitative research workflows.

Infrastructure Category	Software Frameworks Deployed	Role in Quantitative Pipeline
Programming Language	Python (v3.10 and above)	Provides the primary computational runtime for the entire quantitative research pipeline, including data processing, model training, and backtesting execution.
Machine Learning Layer	XGBoost, Scikit-Learn	implements gradient-boosted decision tree models for return prediction, feature importance estimation, and supervised learning-based signal generation.
Data Engineering & Numerical Computation Layer	Pandas, NumPy	Handles high-frequency time-series preprocessing, feature engineering, vectorized transformations, and structured dataset alignment across multiple market intervals.
Visualization & Analytical Interface Layer	Streamlit, Plotly	Enables interactive exploratory analysis of trading strategies, including equity curves, drawdown profiles, signal distributions, and model performance diagnostics.
Market Data Ingestion Layer	Yahoo Finance API, Binance Market Data API	Provides structured historical and real-time market data streams across multiple asset classes, ensuring multi-source consistency for backtesting validation.

7.1 Academic Relevance for Advanced Postgraduate Candidacy

The PBX-Crypto Forecasting Engine provides a practical framework relevant to graduate-level research in **Computational Finance**, **Machine Learning for Finance**, and **Quantitative Risk Modeling**. By addressing key challenges in **financial time-series modeling**, **feature extraction**, and **empirical evaluation**, the project reflects applied understanding of modern machine learning methods in financial contexts.

The repository emphasizes **reproducible research practices** and **structured experimental design**, bridging theoretical modeling approaches with practical implementation in a unified computational pipeline. This combination of methodological design and applied system development reflects competencies commonly valued in research-oriented graduate programs and quantitative finance laboratories.